Student 10 **GONTLA SAI DEEPAK** 192311432RD 2 / 2 submitted

Q1. Smart University Payroll Management System

Score: 0.00 **TC: 0/3 passed**

CODE

```
import java.util.Scanner;

abstract class Employee {
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;

    public Employee() {
    }

    public Employee(int employeeId, String employeeName, String department, double basicSalary) {
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.department = department;
        this.basicSalary = basicSalary;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    public void setEmployeeId(int employeeId) {
        this.employeeId = employeeId;
    }

    public String getEmployeeName() {
        return employeeName;
    }

    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }

    public void displayEmployee() {
        System.out.println("Employee ID : " + employeeId);
        System.out.println("Name : " + employeeName);
        System.out.println("Department : " + department);
        System.out.println("Monthly Salary : " + calculateSalary());
    }

    public abstract double calculateSalary();
}
```

```
class TeachingFaculty extends Employee {
    private double teachingHours;
    private double incentivePerHour;

    public TeachingFaculty(int employeeId, String employeeName, String department, double basicSalary,
                           double teachingHours, double incentivePerHour) {
        super(employeeId, employeeName, department, basicSalary);
        this.teachingHours = teachingHours;
        this.incentivePerHour = incentivePerHour;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (teachingHours * incentivePerHour);
    }
}

class TechnicalStaff extends Employee {
    private double overtimeHours;
    private double overtimeRate;

    public TechnicalStaff(int employeeId, String employeeName, String department, double basicSalary,
                          double overtimeHours, double overtimeRate) {
        super(employeeId, employeeName, department, basicSalary);
        this.overtimeHours = overtimeHours;
        this.overtimeRate = overtimeRate;
    }

    @Override
    public double calculateSalary() {
        return getBasicSalary() + (overtimeHours * overtimeRate);
    }
}

class VisitingFaculty extends Employee {
    private double lecturesHandled;
    private double paymentPerLecture;

    public VisitingFaculty(int employeeId, String employeeName, String department, double basicSalary,
                           double lecturesHandled, double paymentPerLecture) {
        super(employeeId, employeeName, department, basicSalary);
        this.lecturesHandled = lecturesHandled;
        this.paymentPerLecture = paymentPerLecture;
    }

    @Override
    public double calculateSalary() {
        return lecturesHandled * paymentPerLecture;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = Integer.parseInt(sc.nextLine().trim());
        Employee[] employees = new Employee[n];

        for (int i = 0; i < n; i++) {
            String type = sc.nextLine().trim();
            int id = Integer.parseInt(sc.nextLine().trim());
            String name = sc.nextLine().trim();
            String dept = sc.nextLine().trim();
            double basic = Double.parseDouble(sc.nextLine().trim());
```

```
        if (type.equalsIgnoreCase("TeachingFaculty")) {
            double teachingHours = Double.parseDouble(sc.nextLine().trim());
            double incentivePerHour = Double.parseDouble(sc.nextLine().trim());
            employees[i] = new TeachingFaculty(id, name, dept, basic, teachingHours, incentivePerHour);
        } else if (type.equalsIgnoreCase("TechnicalStaff")) {
            double overtimeHours = Double.parseDouble(sc.nextLine().trim());
            double overtimeRate = Double.parseDouble(sc.nextLine().trim());
            employees[i] = new TechnicalStaff(id, name, dept, basic, overtimeHours, overtimeRate);
        } else if (type.equalsIgnoreCase("VisitingFaculty")) {
            double lecturesHandled = Double.parseDouble(sc.nextLine().trim());
            double paymentPerLecture = Double.parseDouble(sc.nextLine().trim());
            employees[i] = new VisitingFaculty(id, name, dept, basic, lecturesHandled, paymentPerLecture);
        }
    }

    System.out.println("Employee Details");
    double total = 0;
    Employee highestPaid = employees[0];

    for (Employee emp : employees) {
        System.out.println();
        emp.displayEmployee();
        total += emp.calculateSalary();
        if (emp.calculateSalary() > highestPaid.calculateSalary()) {
            highestPaid = emp;
        }
    }

    System.out.println();
    System.out.println("Highest Paid Employee");
    System.out.println("Employee ID : " + highestPaid.getEmployeeId());
    System.out.println("Salary : " + highestPaid.calculateSalary());

    System.out.println();
    System.out.println("Average Salary");
    double average = total / employees.length;
    System.out.printf("%.2f%n", average);

    // Search employee
    String searchLine = null;
    while (sc.hasNextLine()) {
        String line = sc.nextLine().trim();
        if (!line.isEmpty()) {
            searchLine = line;
            break;
        }
    }

    if (searchLine != null) {
        int searchId = Integer.parseInt(searchLine);
        System.out.println();
        boolean found = false;
        for (Employee emp : employees) {
            if (emp.getEmployeeId() == searchId) {
                System.out.println("Employee Found");
                System.out.println("Employee ID : " + emp.getEmployeeId());
                System.out.println("Name : " + emp.getEmployeeName());
                System.out.println("Salary : " + emp.calculateSalary());
                found = true;
                break;
            }
        }
        if (!found) {
```

```
                System.out.println("Employee Not Found");
            }
        }
    }
}
```

OUTPUT

Q2. Smart Vehicle Insurance Claim System

Score: 0.00 **TC: 0/3 passed**

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());

        Vehicle[] vehicles = new Vehicle[n];

        for (int i = 0; i < n; i++) {
            String type = sc.nextLine().trim();
            String vehicleNumber = sc.nextLine().trim();
            String ownerName = sc.nextLine().trim();
            double insuredAmount = Double.parseDouble(sc.nextLine().trim());

            if (type.equalsIgnoreCase("Car")) {
                vehicles[i] = new Car(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Bike")) {
                vehicles[i] = new Bike(vehicleNumber, ownerName, insuredAmount);
            } else if (type.equalsIgnoreCase("Truck")) {
                vehicles[i] = new Truck(vehicleNumber, ownerName, insuredAmount);
            }
        }

        // Display all vehicles with claim amounts
        for (Vehicle v : vehicles) {
            System.out.println("Vehicle : " + v.getVehicleNumber());
            System.out.println("Claim Amount : " + v.calculateClaimAmount());
            System.out.println();
        }

        // Find highest claim vehicle
        Vehicle highest = vehicles[0];
        for (Vehicle v : vehicles) {
            if (v.calculateClaimAmount() > highest.calculateClaimAmount()) {
                highest = v;
            }
        }

        System.out.println("Highest Claim Vehicle");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highest.calculateClaimAmount());
        System.out.println();

        // Search by vehicle number
        sc.nextLine(); // consume "Search Vehicle:" line
        String searchNumber = sc.nextLine().trim();

        boolean found = false;
        for (Vehicle v : vehicles) {
            if (v.getVehicleNumber().equals(searchNumber)) {
                System.out.println("Vehicle Found");
                System.out.println("Owner : " + v.getOwnerName());
                System.out.println("Claim Amount : " + v.calculateClaimAmount());
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Vehicle Not Found");
```

```
        }

        sc.close();
}
}

class Vehicle {
    private String vehicleNumber;
    private String ownerName;
    private double insuredAmount;

    public Vehicle(String vehicleNumber, String ownerName, double insuredAmount) {
        this.vehicleNumber = vehicleNumber;
        this.ownerName = ownerName;
        this.insuredAmount = insuredAmount;
    }

    public String getVehicleNumber() { return vehicleNumber; }
    public String getOwnerName() { return ownerName; }
    public double getInsuredAmount() { return insuredAmount; }

    public double calculateClaimAmount() { return 0; }
}

class Car extends Vehicle {
    public Car(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return getInsuredAmount() * 0.80;
    }
}

class Bike extends Vehicle {
    public Bike(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return getInsuredAmount() * 0.60;
    }
}

class Truck extends Vehicle {
    public Truck(String vehicleNumber, String ownerName, double insuredAmount) {
        super(vehicleNumber, ownerName, insuredAmount);
    }

    @Override
    public double calculateClaimAmount() {
        return getInsuredAmount() * 0.90;
    }
}
```

INPUT

```
3
Car
TN10AB1234
Ramesh
800000
```

OUTPUT